

Semantic anonymization

Thanh Huyen Nguyen (12534847) · The Chuong Chu (12534848)

Olesia Galynskaia (12321492) · Kaan Nakipoglu (12502731)

Abstract

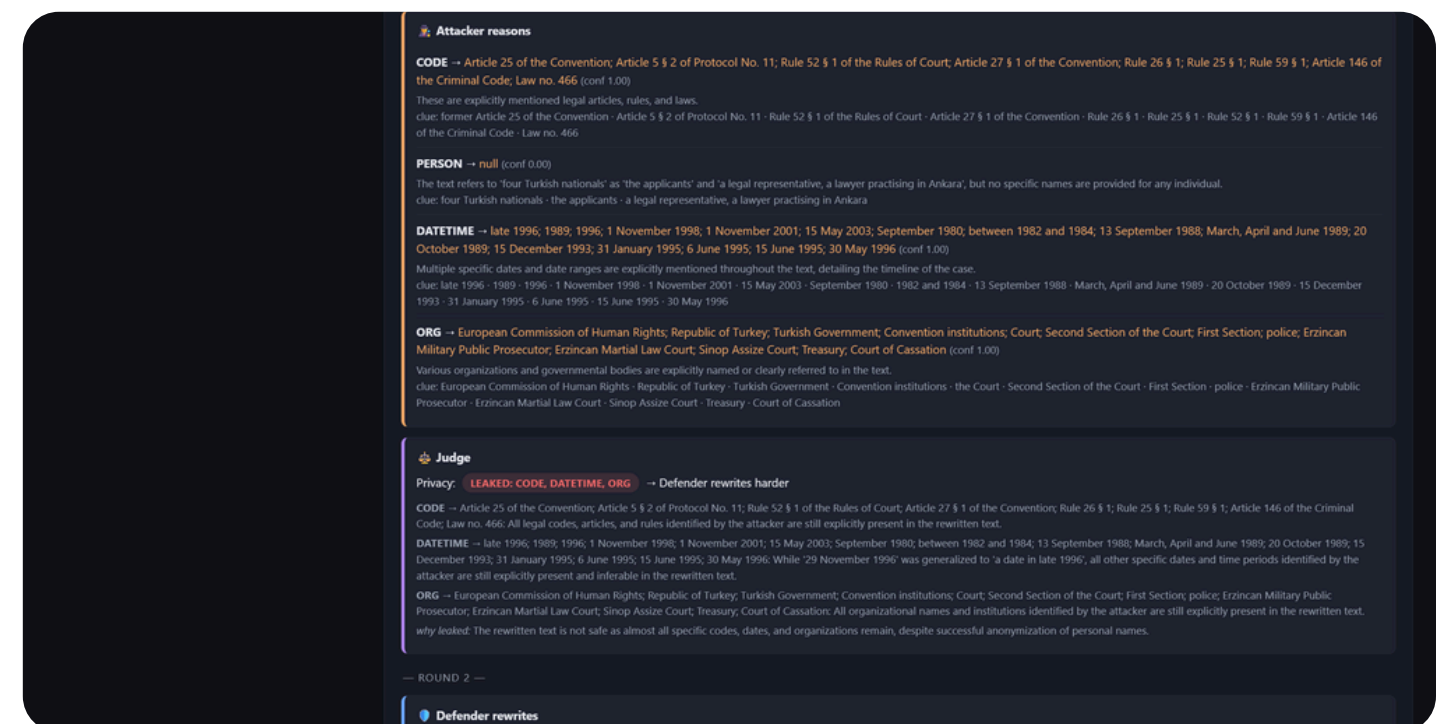
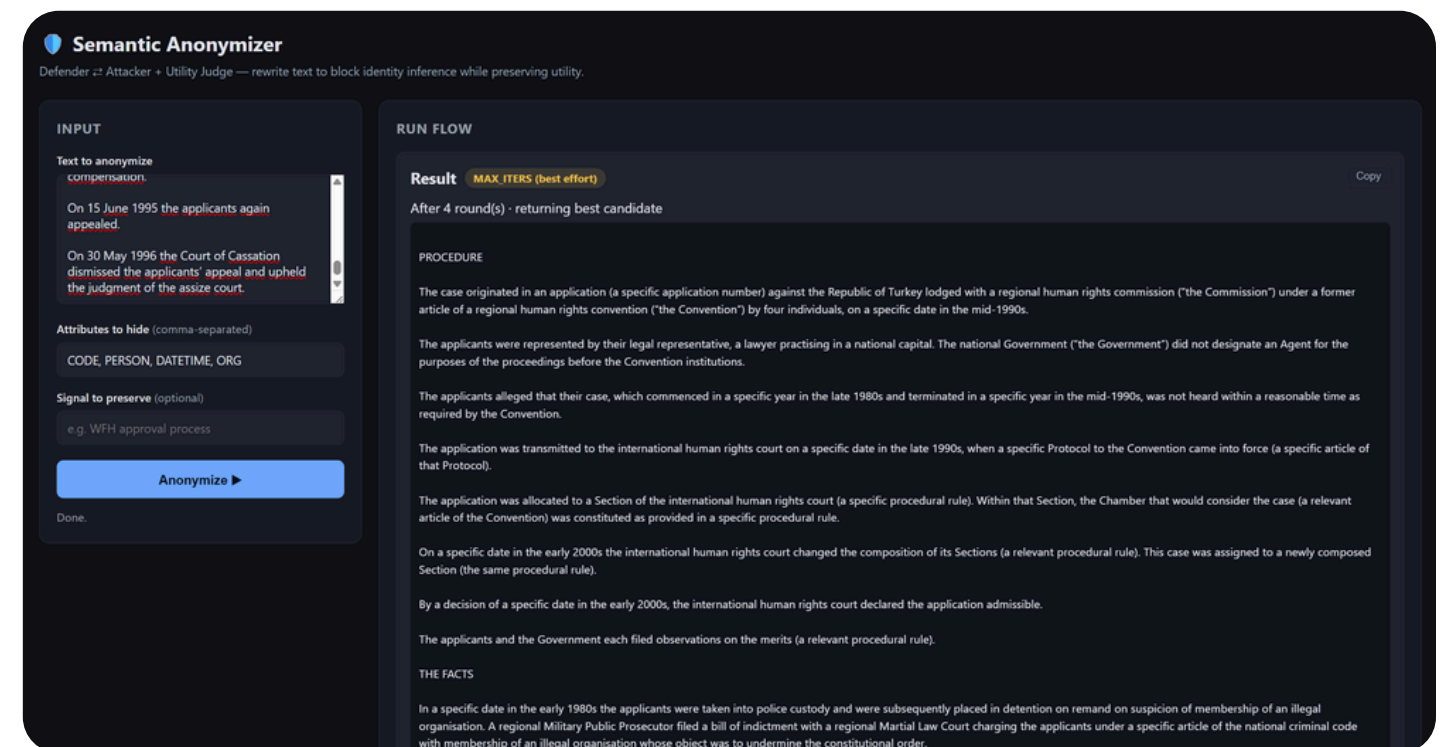
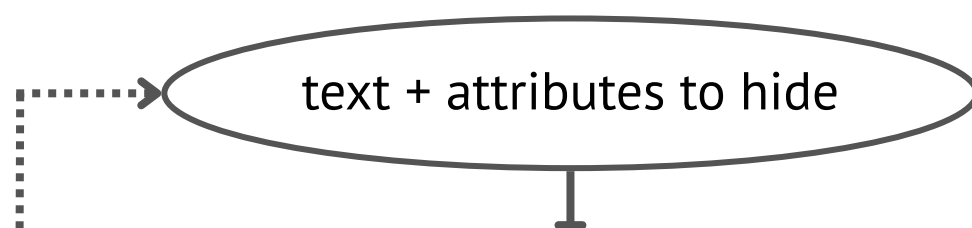
Semantic Anonymizer is a web app where a user pastes text, selects which attributes to hide (e.g. age, profession, location), and gets an anonymized version. Under the hood, three LLM agents – Defender, Attacker, and Judge – run in an iterative loop until the text is both private and useful.

Problem

Standard anonymization tools remove explicit identifiers like names and phone numbers. But personal information often leaks through context and indirect clues.

We wanted to see if an LLM-based adversarial system can do better – and how much utility survives after anonymization.

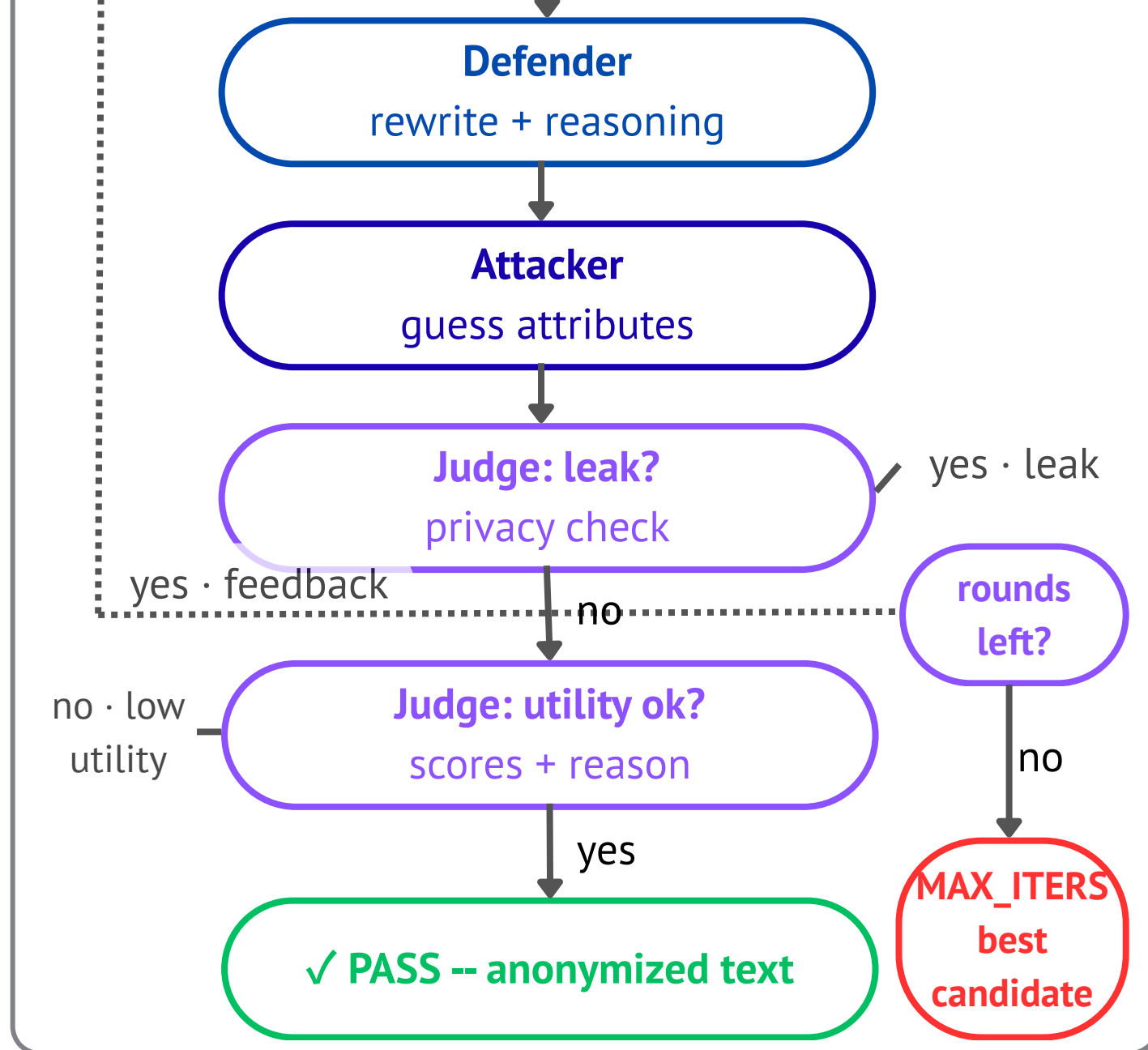
How it works



Evaluation

Use case

As an example of situation to use our app we try legal documents. Court decisions contain sensitive personal information – names, locations, medical details, family circumstances. When such documents are published (in the press, in legal databases, or shared for



Defender	Attacker	Judge
Rewrites text via abstraction, shifting, omission	Infers hidden attributes via chain-of-thought	Two-stage: privacy check -- utility scoring
GLiNER + regex + spaCy pre-scan + feedback	Returns confidence score + evidence spans	Sends targeted feedback back to Defender

research), this information needs to be hidden, but the text still has to make sense.

Dataset

We evaluate on a subset of the Text Anonymization Benchmark (TAB) – 1,268 English court cases from the European Court of Human Rights, manually annotated with entity types, masking decisions (DIRECT / QUASI / NO_MASK), and co-reference relations.

TAB annotations were converted into our format: entities marked DIRECT or QUASI become `attributes_to_hide`, NO_MASK entities stay in the text.

Metrics and Results

56%	9×
privacy success rate (0 leaked attrs)	better than NER baseline (6% → 56% clean)
68%	0.93
pass rate (privacy + utility)	attacker confidence when leak occurs

Utility scores (avg)

Highest leak rate

- Before each rewrite, GLiNER + regex pre-scans for direct identifiers and passes findings as hints to the Defender. On retries, a deterministic verbatim check explicitly flags any spans that survived the previous rewrite.
- The loop runs on LangGraph – each agent is a node, and the graph decides what runs next based on the Judge's verdict.
- All LLM calls go through LangChain, which lets us swap Gemini 2.5 Flash for GPT-4o or a local Ollama model by changing one line in config.
- Each agent operates on a dedicated system prompt: Defender, Attacker, privacy gate, and utility scoring – four prompts in total.
- The backend is FastAPI, streaming results round by round via NDJSON.
- The frontend is plain HTML/JS served by nginx, and the whole thing runs with docker compose up.

Fluency		0.92
Consistency		0.96
Task utility		0.64
Completeness		0.45

DEM	39%
MISC	29%
DATETIME	20%

Learnings and Limitations

- 9× better than NER baseline – 56% clean records vs. 6%. Names and codes are reliably removed.
- Demographic & misc attributes are hardest to hide. Context gives them away even without direct mentions.
- Semantic leaks are confident, not random – Attacker confidence 0.93 on leaked attributes.
- Privacy–utility trade-off – more rounds improve privacy but hurt utility (task: 0.64, completeness: 0.45).
- Most records hit the 4-round cap – more iterations would help, but cost and latency grow fast.
- Research prototype – no API auth; text goes to a third-party LLM. Local Ollama supported for on-premise use.